Glowy

Dossier P5 : Espace client, fidélité, abonnements et PWA

Projet d'équipe · octobre 2026

Ce dossier explique le projet en détail, puis **ta gestion** : ce qu'elle fait, ses règles métier, son IA, les fichiers à connaître et comment la présenter à la validation.

Le projet Glowy en détail

1. Qu'est-ce que Glowy ?

Glowy est une **plateforme de réservation de salons de beauté vérifiés en Tunisie** (coiffure, barbier, ongles, spa, visage…). Une cliente cherche un salon, voit les prestations, les prix en dinars, les avis et l'affluence, puis réserve un créneau précis en moins de 2 minutes. Le salon gère tout son travail depuis un tableau de bord, et un Super Admin contrôle la plateforme.

- **4 rôles** : cliente, employé, gérant de salon, Super Admin.
- **3 langues** : français, arabe (écriture de droite à gauche) et anglais.
- **Application installable** sur téléphone (PWA), utilisable hors connexion.
- **5 intelligences artificielles**, une par membre de l'équipe.
- Projet d'équipe de 5 étudiants : chaque personne a **1 gestion classique + 1 IA**.

2. Les technologies

Domaine	Technologie	Rôle dans le projet
Langage	**TypeScript**	partout : pages, API, règles métier, tests
Framework	**Next.js 16** (App Router) + **React 19**	site et API dans un seul projet
Interface	**Tailwind CSS 4**, composants shadcn / Base UI	mise en page responsive, mobile first
Animations	**GSAP**	apparitions et transitions
Base de données	**MongoDB** + **Mongoose 9**	33 collections (comptes, salons, réservations, avis, points…)
Authentification	**Auth.js v5**	connexion e-mail / mot de passe, Google, sessions sécurisées
Validation	**Zod**	contrôle de toutes les données reçues par l'API
Carte	**Leaflet** + OpenStreetMap	carte des salons

Domaine	Technologie	Rôle dans le projet
Graphiques	**Recharts**	statistiques du dashboard
E-mails	**nodemailer** (SMTP)	mot de passe oublié, rappels
Notifications	**Web Push**	rappels sur le téléphone
Tests	**node:test** + tsx	75 tests automatisés sur une base de test

3. L'architecture

Le code suit une **architecture propre (Clean Architecture)** : chaque domaine est un **module** indépendant dans `src/modules/<module>/`, découpé en 3 couches.

Couche	Contenu	Exemple
`domain`	règles métier pures, sans base de données, faciles à tester	calcul de l'acompte, verrou des créneaux
`application`	cas d'usage et validation des données (Zod)	« créer une réservation »
`infrastructure`	lecture et écriture dans MongoDB	requêtes Mongoose

Organisation des dossiers :

```
src/
  app/              pages et routes API (Next.js)
    (site)/         site public + espace client (/compte)
    (auth)/         connexion, inscription, mot de passe oublié
    dashboard/      espace du salon (gérant, employés)
    admin/          espace Super Admin
    api/            routes REST
  modules/          un dossier par domaine (identity, booking, reviews, loyalty…)
  components/       composants d'interface
  core/domain/      règles communes (erreurs, heure de Tunis, événements)
  infrastructure/   connexion MongoDB, sessions, outils HTTP
  i18n/             traductions français / arabe / anglais
public/sw.js        service worker (hors connexion, notifications)
tests/              tests automatisés
```

Les modules communiquent par événements. Exemple : quand le salon marque un rendez-vous « terminé », le module réservation publie `booking.completed`. Le module fidélité y réagit en créditant les points, et le module notifications prévient la cliente. Aucun module ne modifie le code d'un autre.

Tâches planifiées (toutes les 10 minutes) : rappels de rendez-vous, préavis et expiration des points, analyse des nouveaux avis.

4. La base de données

MongoDB stocke les données sous forme de documents. Les données qui vivent toujours avec leur parent sont **imbriquées** : les horaires et les prestations dans le salon, les lignes et le paiement dans la réservation, le compte fidélité dans l'utilisateur.

Collection	Contenu
`User`	comptes (rôle, mot de passe haché, points, favoris)

Collection	Contenu
`Salon`	salons (adresse, horaires, photos, prestations, plan)
`Employee`, `TimeOff`	équipe et congés
`Booking`	réservations (prestations, statut, acompte)
`SlotLock`	verrou anti double réservation (tranches de 5 min)
`Review`, `ReviewReport`	avis et signalements
`PointTransaction`, `Reward`, `Redemption`	fidélité
`Notification`, `PushSubscription`	notifications
`PlanPayment`	abonnements des salons
`AuditLog`	journal des actions sensibles

5. La sécurité

- Mots de passe hachés (bcrypt) ; compte bloqué 15 minutes après 5 échecs.
- Rôle vérifié sur **chaque page et chaque API**.
- Protection CSRF : une écriture venant d'un autre site est refusée.
- Toutes les données reçues sont validées (Zod) avant d'arriver en base.
- Les actions sensibles du Super Admin sont journalisées.

6. L'application mobile (PWA)

- **Manifest** : nom, icônes, mode plein écran, raccourcis. L'application s'installe sur l'écran d'accueil.
- **Mode hors connexion** : le service worker enregistre les pages consultées et précharge les pages publiques liées. Sans réseau, elles s'ouvrent depuis le téléphone ; sinon la page `/offline` liste les pages disponibles. Les pages personnelles sont effacées à la déconnexion.
- **Mobile first et responsive** : testé de 320 à 1440 px de large, en français, en arabe et en anglais, sans aucun débordement.

7. Lancer le projet

```
npm install
npm run db:seed        données de démonstration
npm run dev            développement : http://localhost:3000
npm run build && npm start      production (nécessaire pour le hors connexion)
npm test               75 tests automatisés
```

Sur la page de connexion, les boutons **« Accès démo par rôle »** connectent en un clic : Client, Gérant, Employé, Super Admin.

8. Les 5 gestions

#	Gestion	Responsable	IA associée
1	Comptes, sécurité et Super Admin	P1	Recommandation de salons
2	Dashboard du salon	P2	Prédiction d'affluence

#	Gestion	Responsable	IA associée
3	Recherche et réservation	P3	Assistante de réservation
4	Avis, modération et support	P4	Analyse des avis
5	Espace client, fidélité, abonnements et PWA	P5	Analyse beauté par photo

Un parcours complet relie les 5 gestions :

1. La cliente se connecte et voit des salons recommandés (gestion 1).
2. Elle réserve un créneau (gestion 3).
3. Le salon voit le rendez-vous dans son agenda et le marque terminé (gestion 2).
4. La cliente gagne des points (gestion 5) et laisse un avis vérifié, analysé par l'IA (gestion 4).
5. Le Super Admin suit l'activité et modère (gestion 1).

5 Gestion de l'espace client, de la fidélité et des abonnements (P5)

But : fidéliser les clientes avec un espace personnel et un programme de points, et gérer les plans payants des salons.

Fonctionnalités

Espace client (`/compte`)

- Profil modifiable.
- Historique des rendez-vous, avec report et **« refaire le même rendez-vous »** en un clic.
- Factures.
- Favoris : salons et professionnels.
- **Notifications** : dans l'application (cloche) et **push** sur le téléphone.
- **E-mails réels** par SMTP quand il est configuré, sinon simulés.

Fidélité — points Glowy

- Points gagnés selon un **barème** réglable par le Super Admin. Par exemple :
  - rendez-vous terminé : +10 ;
  - nouveau salon : +30 ;
  - avis : +15 ;
  - heure creuse : +10 ;
  - parrainage : +50 ;
  - absence : −20.
- **Niveaux** : Bronze, Argent (200 points, +10 % sur les gains), Or (500 points, +20 %), Platine (1 000 points, +30 %), avec barre de progression.
- **Catalogue de récompenses** : un échange donne un code à présenter au salon.
- **Parrainage** avec un code personnel ; badges calculés selon l'activité.
- **Anti-fraude** : plafonds par jour et par mois.
- **Expiration** des points après 12 mois d'inactivité, avec un préavis un mois avant.
- Les points ne sont crédités **qu'après un rendez-vous terminé par le salon**. Chaque mouvement est inscrit dans un grand livre (`PointTransaction`).

Abonnements des salons

- Plans Gratuit, Pro (79 DT) et Premium (149 DT).
- Le gérant déclare son paiement (virement, D17 ou espèces). Le Super Admin l'accepte ou le refuse.
- Le plan accepté est actif **30 jours** et se renouvelle à partir de 7 jours avant l'échéance.
- **Limites appliquées par le serveur** : Gratuit 1 professionnel actif, Pro 8 (avec CRM, statistiques, promotions, vues semaine / mois, prévision d'affluence), Premium illimité (avec stocks, succursales, mise en avant dans la recherche). Une fonction non comprise affiche « Disponible avec le plan … ».
- **Échéance** : rappel au gérant 5 jours avant, puis retour automatique au plan Gratuit (tâche planifiée).
- La page `/pro` présente l'offre aux salons.

Application installable (PWA) et langues

- Installation sur l'écran d'accueil du téléphone.

- **Hors connexion** : les pages déjà vues et les pages publiques liées restent consultables. Les pages personnelles sont effacées à la déconnexion.
- Interface en **français, arabe (de droite à gauche) et anglais**.

IA — Analyse beauté par photo

- **Entrée** : une photo des cheveux ou du visage, avec consentement.
- **Méthode** :
  - mesures calculées **dans le navigateur** à partir des pixels (luminosité, contraste, saturation, reflets, rougeurs, netteté, différence racines/pointes) : **la photo ne quitte jamais l'appareil** ;
  - des **règles explicables** (seuils) en déduisent les points d'attention. Pour les cheveux : terne, frisottis, pointes sèches, racines. Pour le visage : rougeurs, brillance, teint irrégulier, pores, déshydratation.
- **Sortie** :
  - conseils et **vraies prestations** des salons adaptés, dans l'ordre de la recommandation de la gestion 1 ;
  - pour une cliente connectée : le résultat enregistré dans son historique, et +10 points une fois par mois.
- **Message affiché** : « conseil indicatif, pas un diagnostic médical ».
- **Repli** : si la photo est trop sombre ou floue, une nouvelle photo est demandée.
- **Code** : `src/modules/beauty/domain/analysis.ts` · page `/analyse-beaute` · `POST /api/ai/beauty`.

Repères techniques

Modules	`account`, `loyalty`, `notifications`, `subscription`, `beauty`
Pages	`/compte/*`, `/analyse-beaute`, `/pro`, `/dashboard/abonnement`, `/admin/abonnements`, `/offline`
API	`/api/me/*`, `/api/subscriptions`, `/api/admin/payments/*`, `/api/ai/beauty`
Collections	`User` (compte fidélité et favoris imbriqués), `PointRule`, `PointTransaction`, `PointCampaign`, `Reward`, `Redemption`, `PhotoAnalysis`, `Notification`, `PushSubscription`, `OutboxMessage`, `PlanPayment`
Événements écoutés	`booking.completed`, `booking.noShow`, `review.published`, `user.registered`
Tests	`loyalty-reviews`, `loyalty-admin`, `subscription`, `beauty`, `scheduler`

Ma préparation pour la validation

Fichiers à connaître

- `src/modules/account/`, `src/modules/loyalty/`, `src/modules/notifications/`, `src/modules/subscription/`, `src/modules/beauty/`
- Pages : `src/app/(site)/compte/*`, `/analyse-beaute`, `/pro`, `src/app/offline` · API : `src/app/api/me/*`, `src/app/api/subscriptions`
- PWA : `public/sw.js` (service worker), `src/app/manifest.ts`, `src/components/pwa/*`
- Tests : `tests/loyalty-reviews.test.ts`, `tests/loyalty-admin.test.ts`, `tests/subscription.test.ts`, `tests/beauty.test.ts`

Démo (4 minutes)

1. **PWA** (lancer `npm run build && npm start`) :
   - DevTools → Application → Manifest : installable, aucune erreur ;
   - installer l'application ;
   - Network → Offline : les pages déjà vues s'ouvrent, une page jamais vue affiche `/offline` avec la liste des pages disponibles.
2. **Profil** (Read / Update), **favoris** (ajout / retrait), **mes rendez-vous** paginés.
3. **Fidélité** `/compte/fidelite` : solde, niveau, historique, échange d'une récompense (code à présenter au salon).
4. **Abonnement** : le gérant déclare un paiement Pro, le Super Admin l'accepte → plan actif 30 jours.

Montrer l'IA

- Où la montrer : `/analyse-beaute`.

Règles métier à savoir expliquer

- Points crédités **seulement après un rendez-vous terminé par le salon** ; grand livre `PointTransaction`.
- Niveaux Bronze → Argent (200) → Or (500) → Platine (1 000), avec bonus de +10 % à +30 % sur les gains.
- Anti-fraude : plafonds par jour et par mois ; expiration après 12 mois d'inactivité, préavis 1 mois avant.
- PWA : pages personnelles enregistrées sur l'appareil, **effacées à la déconnexion**.

Questions probables du jury

- *Pourquoi le hors ligne ne marche pas en `npm run dev` ?* En développement, Next a besoin du serveur pour démarrer React ; le service worker ne met donc rien en cache.
- *Sur téléphone ?* Installation et hors ligne seulement en HTTPS (ou `localhost`), pas via `http://192.168…`.